

Java 20 — Scoped Values

Java Version: 20

Year: 2023

Feature: Scoped Values — Incubator

UniqueClassName: ScopedValues

Definition

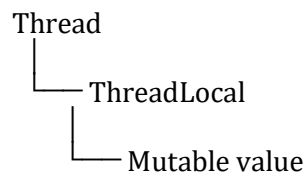
Scoped Values were introduced as an **incubator API in Java 20** as part of **Project Loom**.

They provide a way to share **immutable data from a parent thread or task with child execution**, without relying on ThreadLocal.

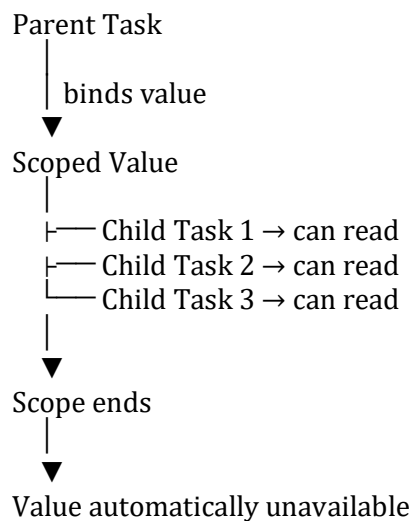
A scoped value has a **bounded lifetime and scope**. A value is made available for a specific operation and is automatically unavailable after that operation finishes.

Basic Idea

With traditional ThreadLocal:

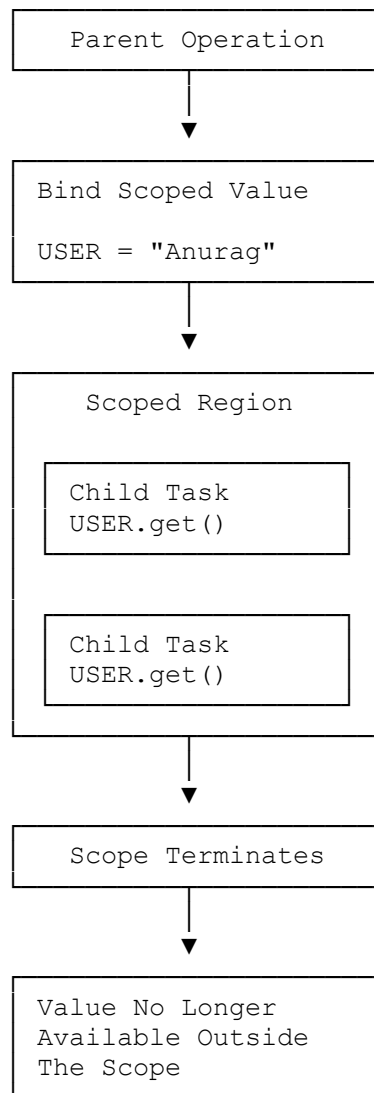


With Scoped Values:



The design is particularly useful with **virtual threads and structured concurrency**.

Flowchart



Java Code Example

Because Scoped Values were an **incubator API in Java 20**, the `jdk.incubator.concurrent` module must be added.

```
import jdk.incubator.concurrent.ScopedValue;

public class ScopedValues {

    private static final ScopedValue<String> USER =
        ScopedValue.newInstance();

    public static void main(String[] args) {
```

```

        ScopedValue.where(USER, "Anurag")
            .run(() -> {

                System.out.println(
                    "Inside scope: " + USER.get()
                );

                processRequest();
            });

        System.out.println(
            "Scope completed."
        );
    }

    private static void processRequest() {

        System.out.println(
            "Processing user: " + USER.get()
        );
    }
}

```

Compile

```

javac --add-modules jdk.incubator.concurrent \
      --enable-preview \
      --release 20 \
      ScopedValues.java

```

Run

```

java --add-modules jdk.incubator.concurrent \
     --enable-preview \
     ScopedValues

```

Output

```

Inside scope: Anurag
Processing user: Anurag
Scope completed.

```

How ScopedValue Works

The key operation is:

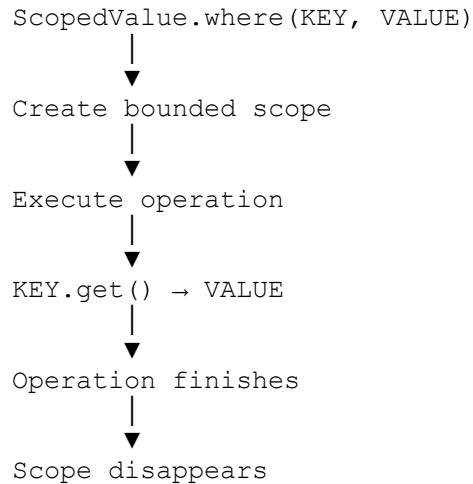
```

ScopedValue.where(USER, "Anurag")
    .run(() -> {
        System.out.println(USER.get());
    });

```

```
});
```

It can be understood as:



Example with Multiple Scoped Values

```
import jdk.incubator.concurrent.ScopedValue;

public class ScopedValues {

    private static final ScopedValue<String> USER =
        ScopedValue.newInstance();

    private static final ScopedValue<String> REQUEST_ID =
        ScopedValue.newInstance();

    public static void main(String[] args) {

        ScopedValue.where(USER, "Anurag")
            .where(REQUEST_ID, "REQ-1001")
            .run(() -> {

                handleRequest();

            });

    }

    private static void handleRequest() {

        System.out.println(
            "User      : " + USER.get()
        );

        System.out.println(
            "Request ID: " + REQUEST_ID.get()
        );
    }
}
```

```
}  
    }  
};
```

Output:

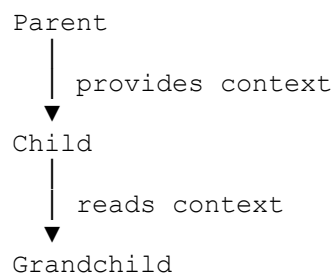
User : Anurag
Request ID : REQ-1001

Scoped Values vs ThreadLocal

Characteristic	ThreadLocal	ScopedValue
Introduced	Java 1.2	Java 20
Java 20 status	Standard API	Incubator
Value model	Mutable by default	Immutable binding
Lifetime	Thread-associated	Lexically/dynamically scoped
Cleanup	Can require explicit management	Scope-based
Data flow	Thread-centric	Execution-scope-centric
Designed for	Thread-local state	Context propagation
Virtual-thread friendly	Can have drawbacks	Designed with Loom in mind

Important Point

A ScopedValue is intended primarily for **one-way context propagation**:



The child does not normally replace the parent's binding globally.

For example:

```
import jdk.incubator.concurrent.ScopedValue;

public class ScopedValues {

    private static final ScopedValue<String> USER =
        ScopedValue.newInstance();

    public static void main(String[] args) {

        ScopedValue.where(USER, "Anurag")
            .run(() -> {

                System.out.println(USER.get());

                ScopedValue.where(USER, "Rahul")
                    .run(() -> {
                        System.out.println(USER.get());
                    });

                System.out.println(USER.get());
            });
    }
}
```

Output:

Anurag
Rahul
Anurag

This demonstrates **nested scopes**:

```
Outer Scope
USER = Anurag
├── Inner Scope
│   USER = Rahul
└── Back to Outer Scope
    USER = Anurag
```

Key Characteristics

Characteristic	Scoped Values
Introduced	Java 20
Year	2023

Status	Incubator
Project	Project Loom
Package	jdk.incubator.concurrent
Main class	ScopedValue
Primary purpose	Context propagation
Mutable shared state	Avoided
Nested scopes	Supported
Useful with virtual threads	Yes
Final feature	Java 25

In short

Scoped Values in Java 20 provide a structured, immutable mechanism for sharing contextual data across a bounded execution scope. They were introduced as an incubator API and were designed particularly with modern concurrency models such as **virtual threads and structured concurrency** in mind.